

Assignment V

1. Finding Unitary Matrix by Hand

Construct the matrix for the transformation from the basis of σ_y to σ_x . In other words, finding the unitary matrix to re-represent a vector in the σ_y eigenvector basis representation in the σ_x eigenvector basis representation.

2. Completeness of Basis

Show that the eigenvectors of σ_x form a complete basis (Hint: see the following equation).

$$\begin{aligned}\sum_{i=0}^{n-1} |i\rangle\langle i| &= |- \rangle\langle -| + | + \rangle\langle +| \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \frac{1}{\sqrt{2}} (1 \quad -1) + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \frac{1}{\sqrt{2}} (1 \quad 1) \\ &= \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I\end{aligned}$$

3. Construction of Matrix

Construct the σ_x by using its eigenvalues and eigenvectors.

4. Colab Verification

Use Colab to verify the results from Problems 1 to 3.

5. Computing Tensor Product

Let $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ and $|g\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix}$, find $|f\rangle_1 \otimes |g\rangle_2$ and $|g\rangle_1 \otimes |f\rangle_2$. Are they the same? Here you see the order is very important.

6. Computing Tensor Products of Three Vectors

Find $\begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. What is the dimension of this space?

7. Distribution Law

Check the following equation:

$$\begin{aligned}(|f\rangle + |e\rangle) \otimes |g\rangle &= |f\rangle \otimes |g\rangle + |e\rangle \otimes |g\rangle \\ |g\rangle \otimes (|f\rangle + |e\rangle) &= |g\rangle \otimes |f\rangle + |g\rangle \otimes |e\rangle\end{aligned}$$

1. Finding Unitary Matrix by Hand

Construct the matrix for the transformation from the basis of σ_y to σ_x . In other words, finding the unitary matrix to re-represent a vector in the σ_y eigenvector basis representation in the σ_x eigenvector basis representation.

$$U |\psi_y\rangle = |\psi_x\rangle$$

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix}$$

Transform from σ_y basis to σ_x basis.

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} (|+\rangle + i|-\rangle)$$

$$|-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix} = \frac{1}{\sqrt{2}} (|+\rangle - i|-\rangle)$$

Construct the Unitary Matrix from inner product:

$$\begin{aligned} U &= \begin{pmatrix} \langle + | + \rangle & \langle + | - \rangle \\ \langle - | + \rangle & \langle - | - \rangle \end{pmatrix} \\ &= \begin{pmatrix} \frac{1+i}{2} & \frac{1-i}{2} \\ \frac{1-i}{2} & \frac{1+i}{2} \end{pmatrix} \end{aligned}$$

$$\therefore U = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad \#$$

2. Completeness of Basis

Show that the eigenvectors of σ_x form a complete basis (Hint: see the following equation).

$$\begin{aligned}\sum_{i=0}^{n-1} |i\rangle\langle i| &= |- \rangle\langle -| + | + \rangle\langle +| \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \frac{1}{\sqrt{2}} (1 \quad -1) + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \frac{1}{\sqrt{2}} (1 \quad 1) \\ &= \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I\end{aligned}$$

Eigenvector

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \begin{matrix} \lambda = +1 \\ |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \end{matrix}, \quad \begin{matrix} \lambda = -1 \\ |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \end{matrix}$$

Completeness Relation:

$$\begin{aligned}\sum_{i=0}^{n-1} |i\rangle\langle i| &= (|-\rangle\langle -|) + (|+\rangle\langle +|) \\ &= \frac{1}{2} \begin{pmatrix} 1 \\ -1 \end{pmatrix} (1 \quad -1) + \frac{1}{2} \begin{pmatrix} 1 \\ 1 \end{pmatrix} (1 \quad 1) \\ &= \frac{1}{2} \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ &= I\end{aligned}$$

$\therefore$ Eigenvectors of σ_x form a Complete basis. *

3. Construction of Matrix

Construct the σ_x by using its eigenvalues and eigenvectors.

Eigen

$$\lambda = +1, \quad |+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\lambda = -1, \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\sigma_x = \lambda_0 |+\rangle \langle +| + \lambda_1 |-\rangle \langle -|$$

$$= (+1) \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \right) + (-1) \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right) \left(\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \end{bmatrix} \right)$$

$$\therefore \sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} *$$

4. Colab Verification

Use Colab to verify the results from Problems 1 to 3.

```
import numpy as np

# ----- Q1: Verify Unitary Matrix -----

# Define the sigma_x and sigma_y eigenvectors
sigma_x_plus = np.array([1 / np.sqrt(2), 1 / np.sqrt(2)]) # |+>
sigma_x_minus = np.array([1 / np.sqrt(2), -1 / np.sqrt(2)]) # |->

sigma_y_plus_i = np.array([1 / np.sqrt(2), 1j / np.sqrt(2)]) # |+i>
sigma_y_minus_i = np.array([1 / np.sqrt(2), -1j / np.sqrt(2)]) # |-i>

# Define the Unitary matrix U for transformation from sigma_y to sigma_x eigenbasis
U = (1 / np.sqrt(2)) * np.array([[1, 1], [1, -1]])

# Verify the transformation of |+i> and |-i> to |+> and |->
result_plus = U @ sigma_y_plus_i
result_minus = U @ sigma_y_minus_i

print("Q1: Verifying Unitary Matrix")
print("U|+i> = ", result_plus)
print("U|-i> = ", result_minus)
print()

# ----- Q2: Verify Completeness of Eigenvectors -----

# Compute the outer products of the eigenvectors of sigma_x
outer_plus = np.outer(sigma_x_plus, sigma_x_plus.conj())
outer_minus = np.outer(sigma_x_minus, sigma_x_minus.conj())

# Compute the sum of the outer products
completeness_check = outer_plus + outer_minus

print("Q2: Verifying Completeness of Eigenvectors")
print("Sum of outer products = \n", completeness_check)
print("Identity matrix = \n", np.eye(2))
print()

# ----- Q3: Verify Construction of sigma_x -----

# Construct sigma_x using its eigenvalues and eigenvectors without scaling by 1/2
sigma_x_constructed = np.outer(sigma_x_plus, sigma_x_plus.conj()) - np.outer(
    sigma_x_minus, sigma_x_minus.conj()
)

# Define the standard sigma_x matrix
sigma_x_standard = np.array([[0, 1], [1, 0]])

print("Q3: Verifying Construction of sigma_x")
print("Constructed sigma_x = \n", sigma_x_constructed)
print("Standard sigma_x = \n", sigma_x_standard)
```

```
• (quantum-computing) (base) pupipatsinghorn@pupipatsinghorn-
tum-computing/.venv/bin/python /Users/pupipatsinghorn/colab/
Q1: Verifying Unitary Matrix
U|+i> = [0.5+0.5j 0.5-0.5j]
U|-i> = [0.5-0.5j 0.5+0.5j]

Q2: Verifying Completeness of Eigenvectors
Sum of outer products =
[[1. 0.]
 [0. 1.]]
Identity matrix =
[[1. 0.]
 [0. 1.]]

Q3: Verifying Construction of sigma_x
Constructed sigma_x =
[[0. 1.]
 [1. 0.]]
Standard sigma_x =
[[0 1]
 [1 0]]
• (quantum-computing) (base) pupipatsinghorn@pupipatsinghorn-
```

5. Computing Tensor Product

Let $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ and $|g\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix}$, find $|f\rangle_1 \otimes |g\rangle_2$ and $|g\rangle_1 \otimes |f\rangle_2$. Are they the same? Here you see the order is very important.

$$|f\rangle_1 \otimes |g\rangle_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \times 1 \\ 0 \times i \\ 1 \times 1 \\ 1 \times i \end{pmatrix}$$

$$\therefore |f\rangle_1 \otimes |g\rangle_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 1 \\ i \end{pmatrix}$$

$$|g\rangle_1 \otimes |f\rangle_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 0 \\ 1 \times 1 \\ i \times 0 \\ i \times 1 \end{pmatrix}$$

$$\therefore |g\rangle_1 \otimes |f\rangle_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 0 \\ i \end{pmatrix}$$

$$\therefore |f\rangle_1 \otimes |g\rangle_2 \neq |g\rangle_1 \otimes |f\rangle_2 \quad *$$

6. Computing Tensor Products of Three Vectors

Find $\begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} \frac{1}{\sqrt{2}} \\ i \\ \frac{1}{\sqrt{2}} \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. What is the dimension of this space?

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \times 1 \\ 0 \times i \\ 1 \times 1 \\ 1 \times i \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 1 \\ i \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \times 0 \\ 0 \times 1 \\ 0 \times 0 \\ 0 \times 1 \\ 1 \times 0 \\ 1 \times 1 \\ i \times 0 \\ i \times 1 \end{pmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ i \end{pmatrix} \quad \#$$

Dimension is 8 #

by substituting $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $|e\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|g\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix}$ to both sides.

8. Tensor Products using Colab

Use Colab to verify the results from Problems 5 to 7. For Problem 5, you may use this code

```
import numpy as np
g = np.array([[0],[1]])
h = np.array([[1/np.sqrt(2)],[1j/np.sqrt(2)]])
print(np.kron(g,h))
print(np.kron(h,g))
```

9. Inner Product of Tensor Products

Check the following equation:

$$\langle h_1 | h_2 \rangle = \left\langle \sum_{j=1}^{n_1} |f_j\rangle \otimes |g_j\rangle \middle| \sum_{l=1}^{n_2} |e_l\rangle \otimes |k_l\rangle \right\rangle = \sum_{j=1}^{n_1} \sum_{l=1}^{n_2} \langle f_j | e_l \rangle \langle g_j | k_l \rangle$$

but setting $n_1 = n_2 = 1$ (i.e. only one term) and $|f\rangle = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $|e\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $|g\rangle = \begin{pmatrix} i \\ i \end{pmatrix}$, and $|k\rangle = \begin{pmatrix} i \\ 1 \end{pmatrix}$.

10. Hand Calculation of Tensor Product

Let $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ and $|g\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{i}{\sqrt{2}} \end{pmatrix}$, find $\sigma_x |f\rangle_1$ and $\sigma_x |g\rangle_2$. Then find the tensor product of the resulting vector (i.e. $\sigma_x |f\rangle_1 \otimes \sigma_x |g\rangle_2$). Now find $|f\rangle_1 \otimes |g\rangle_2$ and construct $\sigma_x \otimes \sigma_x$. Show that $\sigma_x \otimes \sigma_x (|f\rangle_1 \otimes |g\rangle_2)$ gives the same result.

11. Tensor Product Calculation using Colab

Use Colab and the function np.kron to verify Problem 10.

7. Distribution Law

Check the following equation:

$$(|f\rangle + |e\rangle) \otimes |g\rangle = |f\rangle \otimes |g\rangle + |e\rangle \otimes |g\rangle$$

$$|g\rangle \otimes (|f\rangle + |e\rangle) = |g\rangle \otimes |f\rangle + |g\rangle \otimes |e\rangle$$

by substituting $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $|e\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $|g\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}$ to both sides.

I.

$$(|f\rangle + |e\rangle) \otimes |g\rangle \stackrel{?}{=} |f\rangle \otimes |g\rangle + |e\rangle \otimes |g\rangle$$

$$\left(\begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right) \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \stackrel{?}{=} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}$$

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 1 \\ 1 \times i \\ 1 \times 1 \\ 1 \times i \end{pmatrix} \stackrel{?}{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \times 1 \\ 0 \times i \\ 1 \times 1 \\ 1 \times i \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 1 \\ 1 \times i \\ 0 \times 1 \\ 0 \times i \end{pmatrix}$$

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \\ 1 \\ i \end{pmatrix} \stackrel{?}{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \\ 1 \\ i \end{pmatrix}$$

True.

II.

$$|g\rangle \otimes (|f\rangle + |e\rangle) \stackrel{?}{=} |g\rangle \otimes |f\rangle + |g\rangle \otimes |e\rangle$$

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \otimes \left(\begin{pmatrix} 0 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right) \stackrel{?}{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} \otimes \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 1 \\ 1 \times i \\ i \times 1 \\ i \times 1 \end{pmatrix} \stackrel{?}{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 0 \\ 1 \times 1 \\ i \times 0 \\ i \times 1 \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times 1 \\ 1 \times 0 \\ i \times 1 \\ i \times 0 \end{pmatrix}$$

$$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \\ i \\ i \end{pmatrix} \stackrel{?}{=} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \\ i \\ i \end{pmatrix}$$

True.

8. Tensor Products using Colab

Use Colab to verify the results from Problems 5 to 7. For Problem 5, you may use this code

```
import numpy as np
g = np.array([[0],[1]])
h = np.array([[1/np.sqrt(2)],[1j/np.sqrt(2)]])
print(np.kron(g,h))
print(np.kron(h,g))
```

Q8 - 5.

```
import numpy as np

g = np.array([[0], [1]])
h = np.array([[1 / np.sqrt(2)], [1j / np.sqrt(2)]])
print(np.kron(g, h))
print(np.kron(h, g))
print(np.kron(g, h) == np.kron(h, g))
```

```
● (quantum-computing) (base) pup
tum-computing/.venv/bin/python
[[0.      +0.j      ]
 [0.      +0.j      ]
 [0.70710678+0.j    ]
 [0.      +0.70710678j]]
[[0.      +0.j      ]
 [0.70710678+0.j    ]
 [0.      +0.j      ]
 [0.      +0.70710678j]]
[[ True]
 [False]
 [False]
 [ True]]
○ (quantum-computing) (base) pup
```

Q8-6.

```
import numpy as np

# Define the three vectors
f = np.array([0, 1]) # |f>
g = (1 / np.sqrt(2)) * np.array([1, 1j]) # |g>
h = np.array([0, 1]) # |h>

# Compute the tensor product of the three vectors
tensor_f_g = np.kron(f, g) # |f> ⊗ |g>
tensor_f_g_h = np.kron(tensor_f_g, h) # (|f> ⊗ |g>) ⊗ |h>

# Print the result and the dimension of the resulting space
print("Q6: Verifying Tensor Product of Three Vectors")
print("Tensor product |f> ⊗ |g> ⊗ |h> = \n", tensor_f_g_h)
print("Dimension of the resulting space: ", tensor_f_g_h.shape[0])
```

```
● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air
tum-computing/.venv/bin/python /Users/pupipatsingkhorn/Developer/
Q6: Verifying Tensor Product of Three Vectors
Tensor product |f> ⊗ |g> ⊗ |h> =
[[0.      +0.j      0.      +0.j      0.      +0.j
 0.      +0.j      0.      +0.j      0.70710678+0.j
 0.      +0.j      0.      +0.70710678j]]
Dimension of the resulting space: 8
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air
```

Q8-7.

```
import numpy as np

# Define the quantum states |f>, |g>, and |h>
f = np.array([0, 1]) # |f>
g = np.array([1, 0]) # |g>
h = np.array([0, 1]) # |h>

# Compute the tensor products
lhs_1 = np.kron(f, (g + h)) # |f> ⊗ (|g> + |h>)
rhs_1 = np.kron(f, g) + np.kron(f, h) # |f> ⊗ |g> + |f> ⊗ |h>

lhs_2 = np.kron((f + g), h) # (|f> + |g>) ⊗ |h>
rhs_2 = np.kron(f, h) + np.kron(g, h) # |f> ⊗ |h> + |g> ⊗ |h>

# Print the results and check if the two sides are the same
print("Q7: Verifying Distribution Laws")

# Law 1: |f> ⊗ (|g> + |h>) = |f> ⊗ |g> + |f> ⊗ |h>
print("Law 1: |f> ⊗ (|g> + |h>) = |f> ⊗ |g> + |f> ⊗ |h>")
print("Left-hand side (|f> ⊗ (|g> + |h>)) = ", lhs_1)
print("Right-hand side (|f> ⊗ |g> + |f> ⊗ |h>) = ", rhs_1)

# Law 2: (|f> + |g>) ⊗ |h> = |f> ⊗ |h> + |g> ⊗ |h>
print("\nLaw 2: (|f> + |g>) ⊗ |h> = |f> ⊗ |h> + |g> ⊗ |h>")
print("Left-hand side ((|f> + |g>) ⊗ |h>) = ", lhs_2)
print("Right-hand side (|f> ⊗ |h> + |g> ⊗ |h>) = ", rhs_2)
```

```
● (quantum-computing) (base) pupipatsinghorn@Pupipats-Ma
tum-computing/.venv/bin/python /Users/pupipatsinghorn/
Q7: Verifying Distribution Laws
Law 1: |f> ⊗ (|g> + |h>) = |f> ⊗ |g> + |f> ⊗ |h>
Left-hand side (|f> ⊗ (|g> + |h>)) = [0 0 1 1]
Right-hand side (|f> ⊗ |g> + |f> ⊗ |h>) = [0 0 1 1]

Law 2: (|f> + |g>) ⊗ |h> = |f> ⊗ |h> + |g> ⊗ |h>
Left-hand side ((|f> + |g>) ⊗ |h>) = [0 1 0 1]
Right-hand side (|f> ⊗ |h> + |g> ⊗ |h>) = [0 1 0 1]
○ (quantum-computing) (base) pupipatsinghorn@Pupipats-Ma
```

9. Inner Product of Tensor Products

Check the following equation:

$$\langle h_1 | h_2 \rangle = \left\langle \sum_{j=1}^{n_1} |f_j\rangle \otimes |g_j\rangle \left| \sum_{l=1}^{n_2} |e_l\rangle \otimes |k_l\rangle \right. \right\rangle = \sum_{j=1}^{n_1} \sum_{l=1}^{n_2} \langle f_j | e_l \rangle \langle g_j | k_l \rangle$$

but setting $n_1 = n_2 = 1$ (i.e. only one term) and $|f\rangle = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$, $|e\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $|g\rangle = \begin{pmatrix} i \\ i \end{pmatrix}$, and $|k\rangle = \begin{pmatrix} i \\ 1 \end{pmatrix}$.

```
import numpy as np

# Define the quantum states |f>, |e>, |g>, and |k>
f = np.array([1, 1]) # |f>
e = np.array([0, 1]) # |e>
g = np.array([1j, 1j]) # |g>
k = np.array([1j, 1]) # |k>

# Set n1 = n2 = 1 as given in the problem
n1 = 1
n2 = 1

# Compute the tensor products
tensor_f_g = np.kron(f, g) # |f> ⊗ |g>
tensor_e_k = np.kron(e, k) # |e> ⊗ |k>

# Compute the inner product of tensor products
inner_product = np.dot(tensor_f_g.conj(), tensor_e_k) # <f| ⊗ <g| and |e> ⊗ |k>

# Print the result
print("Q9: Verifying Inner Product of Tensor Products")
print("Inner product <f| ⊗ <g| and |e> ⊗ |k> = ", inner_product)
```

```
● (quantum-computing) (base) pupipatsingkhorn@Pupi
tum-computing/.venv/bin/python /Users/pupipatsin
Q9: Verifying Inner Product of Tensor Products
Inner product <f| ⊗ <g| and |e> ⊗ |k> = (1-1j)
○ (quantum-computing) (base) pupipatsingkhorn@Pupi
```

#

10. Hand Calculation of Tensor Product

Let $|f\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ and $|g\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ i \end{pmatrix}$, find $\sigma_x|f\rangle_1$ and $\sigma_x|g\rangle_2$. Then find the tensor product of the resulting vector (i.e. $\sigma_x|f\rangle_1 \otimes \sigma_x|g\rangle_2$). Now find $|f\rangle_1 \otimes |g\rangle_2$ and construct $\sigma_x \otimes \sigma_x$. Show that $\sigma_x \otimes \sigma_x (|f\rangle_1 \otimes |g\rangle_2)$ gives the same result.

$$\sigma_x |f\rangle_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad *$$

$$\sigma_x |g\rangle_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} i \\ 1 \end{pmatrix} \quad *$$

$$\therefore \sigma_x |f\rangle_1 \otimes \sigma_x |g\rangle_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} i \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \times i \\ 1 \times 1 \\ 0 \times i \\ 0 \times 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} i \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad *$$

$$|f\rangle_1 \otimes |g\rangle_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \times 1 \\ 0 \times i \\ 1 \times 1 \\ 1 \times i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 1 \\ i \end{pmatrix} \quad *$$

$$\sigma_x \otimes \sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \otimes \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \cdot \sigma_x & 1 \cdot \sigma_x \\ 1 \cdot \sigma_x & 0 \cdot \sigma_x \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \quad *$$

$\therefore$ Same *

$$\therefore \sigma_x \otimes \sigma_x (|f\rangle_1 \otimes |g\rangle_2) = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} i \\ 1 \\ 0 \\ 0 \end{pmatrix} \quad *$$

11. Tensor Product Calculation using Colab

Use Colab and the function `np.kron` to verify Problem 10.

11. Tensor Product Calculation using Colab

Use Colab and the function `np.kron` to verify Problem 10.

```

import numpy as np

# 1. Tensor Product of Operators
# Define two operators A and B (Example: Pauli-X and Pauli-Z matrices)
A = np.array([[0, 1], [1, 0]]) # Pauli-X operator (sigma_x)
B = np.array([[1, 0], [0, -1]]) # Pauli-Z operator (sigma_z)

# Compute the tensor product of operators A and B
tensor_product = np.kron(A, B)
print("Tensor product of A and B (A ⊗ B):")
print(tensor_product)
print()

# 2. Partial Measurement (Partial Trace)
# We want to compute the partial trace of a composite system described by the tensor product
# of operators A and B.
# In this case, we compute the partial trace over subsystem B (trace over the second
# subsystem).

# Function to compute the partial trace of a tensor product
def partial_trace(tensor):
    # Compute partial trace over the second subsystem (B)
    # For a 2x2 system, we trace out the second subsystem, reducing the 4x4 matrix to a 2x2
    # matrix.
    # The matrix is 4x4, so we want to sum over the diagonal elements that correspond to
    # subsystem A.
    # We'll iterate through the 2x2 blocks in the 4x4 matrix and take the trace of each block.
    # Initialize partial trace result
    trace_result = np.zeros((2, 2))

    # Compute partial trace by summing over the diagonal blocks
    for i in range(2):
        for j in range(2):
            trace_result[i, j] = tensor[i * 2, j * 2] + tensor[i * 2 + 1, j * 2 + 1]

    return trace_result

# Compute partial trace over subsystem B (trace over the second subsystem)
partial_trace_result = partial_trace(tensor_product)
print("Partial trace over subsystem B:")
print(partial_trace_result)
print()

# 3. Matrix Representation of an Operator in a Given Basis
# Define the computational basis states |0> and |1>
basis_0 = np.array([1, 0]) # |0>
basis_1 = np.array([0, 1]) # |1>

# Now, let's compute the matrix elements of the operator A (Pauli-X) in the computational
# basis
O_matrix = np.array(
    [
        [np.dot(basis_0.T, np.dot(A, basis_0)), np.dot(basis_0.T, np.dot(A, basis_1))],
        [np.dot(basis_1.T, np.dot(A, basis_0)), np.dot(basis_1.T, np.dot(A, basis_1))],
    ]
)

print("Matrix representation of operator A (Pauli-X) in the computational basis:")
print(O_matrix)
print()

# 4. Boolean Comparison to Check if Partial Trace and Tensor Product Give the Same Result
# Check if partial trace and tensor product match
comparison_result = np.allclose(partial_trace_result, tensor_product[:2, :2])

print("Do the partial trace and tensor product give the same result?")
print(comparison_result)

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```

● (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True
○ (quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing
  └─$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
    Tensor product of A and B ( $A \otimes B$ ):
    [[ 0  0  1  0]
     [ 0  0  0 -1]
     [ 1  0  0  0]
     [ 0 -1  0  0]]

    Partial trace over subsystem B:
    [[0. 0.]
     [0. 0.]]

    Matrix representation of operator A (Pauli-X) in the computational basis:
    [[0 1]
     [1 0]]

    Do the partial trace and tensor product give the same result?
    True

```

```
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
```

Tensor product of A and B ($A \otimes B$):

```
[[ 0  0  1  0]
 [ 0  0  0 -1]
 [ 1  0  0  0]
 [ 0 -1  0  0]]
```

Partial trace over subsystem B:

```
[[0. 0.]
 [0. 0.]]
```

Matrix representation of operator A (Pauli-X) in the computational basis:

```
[[0 1]
 [1 0]]
```

Do the partial trace and tensor product give the same result?

```
True
```

```
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing$
```

```
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
```

Tensor product of A and B ($A \otimes B$):

```
[[ 0  0  1  0]
 [ 0  0  0 -1]
 [ 1  0  0  0]
 [ 0 -1  0  0]]
```

Partial trace over subsystem B:

```
[[0. 0.]
 [0. 0.]]
```

Matrix representation of operator A (Pauli-X) in the computational basis:

```
[[0 1]
 [1 0]]
```

Do the partial trace and tensor product give the same result?

```
True
```

```
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing$
```

```
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-computing$ python /Users/pupipatsingkhorn/Developer/repositories/quantum-computing/.venv/bin/python
```

```
Tensor product of A and B ( $A \otimes B$ ):  
[[ 0  0  1  0]  
 [ 0  0  0 -1]  
 [ 1  0  0  0]  
 [ 0 -1  0  0]]  
  
Partial trace over subsystem B:  
[[0. 0.]  
 [0. 0.]]  
  
Matrix representation of operator A (Pauli-X) in the computational basis:  
[[0 1]  
 [1 0]]  
  
Do the partial trace and tensor product give the same result?  
True  
(quantum-computing) (base) pupipatsingkhorn@Pupipats-MacBook-Air quantum-com-
```